

INF672 / Unidad 1 / GL02

## GUÍA DE LABORATORIO

# Introducción a NumPy, Pandas y Matplotlib

*Las herramientas de datos que usaremos en toda la materia, sobre un mismo dataset: iris.*

Machine Learning · 6º semestre · Gestión 2026-02

Duración: 3 h de laboratorio

UATF · Ing. Informática · Docente: M.Sc. Huáscar Fedor Gonzales Guzmán

**NOTA Convenciones de esta guía****Bloques oscuros** = terminal. **In [ ] / Out[ ]** = celdas de Jupyter con su código y su resultado.

Reproduce cada celda en tu cuaderno.

## 01 Competencia y objetivos

### Competencia de la práctica

Manipula datos numéricos y tabulares con NumPy y Pandas, y comunica hallazgos mediante visualizaciones con Matplotlib y Seaborn, siguiendo el flujo de trabajo básico del análisis de datos que sostiene todo proyecto de Machine Learning.

### Objetivos específicos

- Crear y operar arreglos de NumPy con operaciones vectorizadas y funciones de agregación.
- Construir y explorar Series y DataFrames de Pandas, y cargar datos desde archivos con `read_csv`.
- Usar el dataset iris como caso de estudio recurrente a lo largo de la materia.
- Generar gráficos con Matplotlib (dispersión, histograma, barras y subgráficos).
- Producir visualizaciones exploratorias rápidas con Seaborn.

## 02 Requisitos y preparación

Esta práctica continúa la GL01. Necesitas el entorno inf672 con las librerías ya instaladas (numpy, pandas, matplotlib, seaborn, scikit-learn).

1. Lanza Jupyter Notebook con el entorno inf672 (desde Anaconda Navigator o desde la terminal).
2. Crea un cuaderno nuevo: New ▶ Notebook ▶ Python 3, y guárdalo como GL02.
3. Escribe en la primera celda los imports que usaremos en toda la guía.

**In [1]:**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

**NOTA**

Los alias np, pd, plt y sns son la convención universal: verás el mismo estilo en la documentación oficial, en libros y en la bibliografía de la materia. Mantenlos siempre.

## 03 NumPy — el arreglo n-dimensional

**DEFINICIÓN ndarray**

El ndarray (arreglo n-dimensional) es la estructura central de NumPy: una colección de elementos del mismo tipo dispuestos en una o más dimensiones. A diferencia de una lista de Python, permite operaciones vectorizadas (aplicar una operación a todos los elementos sin bucles), lo que lo hace mucho más rápido y conciso.

### 3.1 Crear arreglos

In [2]:

```
a = np.array([2, 4, 6, 8])
a
```

Out[2]:

```
array([2, 4, 6, 8])
```

In [3]:

```
np.arange(0, 10, 2)      # inicio, fin (excl.), paso
```

Out[3]:

```
array([0, 2, 4, 6, 8])
```

In [4]:

```
np.linspace(0, 1, 5)     # 5 valores equiespaciados en [0, 1]
```

Out[4]:

```
array([0. , 0.25, 0.5 , 0.75, 1.  ])
```

**NOTA**

np.zeros(n), np.ones(n) y np.random.rand(n) también crean arreglos: ceros, unos y valores aleatorios. Son útiles para inicializar o generar datos de prueba.

### 3.2 Forma, tipo y atributos

Todo arreglo conoce su forma (shape), su tipo de dato (dtype) y su número de dimensiones. Un arreglo 2D es una matriz de filas y columnas:

In [5]:

```
M = np.array([[1, 2, 3],
               [4, 5, 6]])
M.shape, M.dtype, M.ndim
```

Out[5]:

```
((2, 3), dtype('int64'), 2)
```

### 3.3 Indexado y slicing

Se accede a los elementos por posición (empezando en 0). En 2D, el primer índice es la fila y el segundo la columna; los dos puntos (:) seleccionan todo un eje:

In [6]:

```
M[0]          # primera fila
```

Out[6]:

```
array([1, 2, 3])
```

In [7]:

```
M[:, 1]       # segunda columna (todas las filas)
```

Out[7]:

```
array([2, 5])
```

### 3.4 Operaciones vectorizadas y agregación

Las operaciones se aplican elemento a elemento sobre todo el arreglo, sin escribir bucles. Las funciones de agregación resumen los datos, y con axis se agregan por filas o por columnas:

In [8]:

```
a * 10         # multiplica cada elemento
```

Out[8]:

```
array([20, 40, 60, 80])
```

In [9]:

```
a.mean(), a.sum(), a.std()
```

Out[9]:

```
(5.0, 20, 2.23606797749979)
```

In [10]:

```
M.sum(axis=0)  # suma por columnas (recorre las filas)
```

Out[10]:

```
array([5, 7, 9])
```

#### NOTA ¿Por qué vectorizar?

Vectorizar no es solo más corto: NumPy ejecuta esas operaciones en código optimizado por debajo, mucho más rápido que un bucle de Python. En Machine Learning, donde se opera sobre miles de datos a la vez, esta diferencia es decisiva.

### 3.5 Del arreglo al dataset real: iris

A partir de aquí trabajaremos con iris, un dataset clásico de 150 flores descritas por 4 medidas (largo y ancho de sépalo y pétalo). Viene incluido en scikit-learn y sus datos son, precisamente, un arreglo de NumPy:

In [11]:

```
from sklearn.datasets import load_iris

iris = load_iris()
iris.data.shape      # 150 muestras, 4 características
```

Out[11]:

```
(150, 4)
```

In [12]:

```
iris.data.mean(axis=0)  # media de cada característica
```

Out[12]:

```
array([5.843, 3.054, 3.759, 1.199])
```

#### NOTA

Aquí iris.data es un ndarray puro: útil para el cálculo, pero sin nombres de columnas ni etiquetas legibles. Ese es justamente el trabajo de Pandas, que veremos a continuación.

## 04 Pandas — Series

### DEFINICIÓN Series

Una Series es un arreglo unidimensional etiquetado: como una columna de una tabla, donde cada valor tiene asociada una etiqueta (su índice). Combina la velocidad de NumPy con un índice que da significado a cada dato.

### 4.1 Crear una Series

In [13]:

```
s = pd.Series([5.1, 4.9, 4.7, 4.6],
               index=["m1", "m2", "m3", "m4"])
s
```

Out[13]:

```
m1    5.1
m2    4.9
m3    4.7
m4    4.6
dtype: float64
```

## 4.2 Acceso, filtrado y métodos

Se accede a los valores por etiqueta o por posición, se filtran con condiciones booleanas y se resumen con métodos estadísticos:

In [14]:

```
s["m1"], s.iloc[0]      # por etiqueta y por posición
```

Out[14]:

```
(5.1, 5.1)
```

In [15]:

```
s[s > 4.7]              # filtrado booleano
```

Out[15]:

```
m1    5.1
m2    4.9
dtype: float64
```

In [16]:

```
s.mean(), s.max()
```

Out[16]:

```
(4.825, 5.1)
```

## 05 Pandas — DataFrames

### DEFINICIÓN DataFrame

Un DataFrame es una tabla bidimensional: un conjunto de columnas (cada una es una Series) que comparten el mismo índice de filas. Es la estructura que más usarás: representa un dataset completo, con sus filas (muestras) y columnas (características).

### 5.1 Crear un DataFrame

In [17]:

```
pd.DataFrame({
    "nombre": ["Ana", "Beto", "Carla"],
    "nota":   [89, 74, 95],
})
```

Out[17]:

```
   nombre  nota
0     Ana    89
1     Beto    74
2     Carla   95
```

### 5.2 Cargar el dataset iris

Construimos un DataFrame con los datos de iris. Usamos nombres de columna cortos y agregamos la especie como columna categórica:

In [18]:

```
from sklearn.datasets import load_iris
iris = load_iris()

df = pd.DataFrame(iris.data,
                  columns=["sepal_len", "sepal_wid",
                          "petal_len", "petal_wid"])
df["species"] = pd.Categorical.from_codes(iris.target,
                                          iris.target_names)

df.head()
```

Out[18]:

|   | sepal_len | sepal_wid | petal_len | petal_wid | species |
|---|-----------|-----------|-----------|-----------|---------|
| 0 | 5.1       | 3.5       | 1.4       | 0.2       | setosa  |
| 1 | 4.9       | 3.0       | 1.4       | 0.2       | setosa  |
| 2 | 4.7       | 3.2       | 1.3       | 0.2       | setosa  |
| 3 | 4.6       | 3.1       | 1.5       | 0.2       | setosa  |
| 4 | 5.0       | 3.6       | 1.4       | 0.2       | setosa  |

### 5.3 Explorar el DataFrame

Antes de analizar, conviene mirar la forma y un resumen de los datos. `head()` muestra las primeras filas; `shape` da (filas, columnas); `describe()` resume estadísticamente las columnas numéricas:

In [19]:

```
df.shape
```

Out[19]:

```
(150, 5)
```

In [20]:

```
df["sepal_len"].describe()
```

Out[20]:

```
count    150.000000
mean       5.843333
std        0.828066
min        4.300000
25%        5.100000
50%        5.800000
75%        6.400000
max        7.900000
Name: sepal_len, dtype: float64
```

### 5.4 Seleccionar y filtrar

Se selecciona una columna con corchetes y se filtran filas con una condición booleana, igual que en NumPy pero conservando etiquetas:

In [21]:

```
df[df["species"] == "setosa"].head(3)    # solo setosa
```

Out[21]:

|   | sepal_len | sepal_wid | petal_len | petal_wid | species |
|---|-----------|-----------|-----------|-----------|---------|
| 0 | 5.1       | 3.5       | 1.4       | 0.2       | setosa  |
| 1 | 4.9       | 3.0       | 1.4       | 0.2       | setosa  |
| 2 | 4.7       | 3.2       | 1.3       | 0.2       | setosa  |

## 5.5 Columnas derivadas y groupby

Podemos crear columnas nuevas a partir de las existentes, y agrupar por una categoría para resumir cada grupo. groupby es una de las operaciones más potentes de Pandas:

In [22]:

```
df["petal_ratio"] = df["petal_len"] / df["petal_wid"]
df[["petal_len", "petal_wid", "petal_ratio"]].head(3)
```

Out[22]:

|   | petal_len | petal_wid | petal_ratio |
|---|-----------|-----------|-------------|
| 0 | 1.4       | 0.2       | 7.00        |
| 1 | 1.4       | 0.2       | 7.00        |
| 2 | 1.3       | 0.2       | 6.50        |

In [23]:

```
df.groupby("species")[["sepal_len", "petal_len"]].mean()
```

Out[23]:

|            | sepal_len | petal_len |
|------------|-----------|-----------|
| species    |           |           |
| setosa     | 5.006     | 1.462     |
| versicolor | 5.936     | 4.260     |
| virginica  | 6.588     | 5.552     |

### EJEMPLO

Ya se ve un patrón clave: setosa tiene pétalos mucho más pequeños que las otras dos especies. Ese tipo de diferencia entre grupos es lo que después permitirá a un modelo distinguir las clases.

## 5.6 Cargar datos desde un archivo (pd.read\_csv)

En la práctica, los datos casi nunca vienen de una librería: llegan en archivos. read\_csv es la puerta de entrada de la mayoría de los proyectos. Guardemos iris a CSV y leámoslo de vuelta:

In [24]:

```
df.to_csv("iris.csv", index=False) # guardar a archivo
datos = pd.read_csv("iris.csv")    # leer desde archivo
datos.head(2)
```

Out[24]:

|   | sepal_len | sepal_wid | petal_len | petal_wid | species | petal_ratio |
|---|-----------|-----------|-----------|-----------|---------|-------------|
| 0 | 5.1       | 3.5       | 1.4       | 0.2       | setosa  | 7.00        |
| 1 | 4.9       | 3.0       | 1.4       | 0.2       | setosa  | 7.00        |

## NOTA

`read_csv` acepta parámetros muy útiles: `sep` para el separador, `header` y `names` para los encabezados, `index_col` para elegir la columna índice y `na_values` para marcar datos faltantes. Volverás a él en casi todas las prácticas.

## 06 Matplotlib — visualización

Matplotlib es la librería base de gráficos en Python. El patrón habitual es: dibujar con una función (`plt.scatter`, `plt.hist`...), etiquetar los ejes y el título, y cerrar con `plt.show()`. Trabajaremos sobre el `DataFrame` `df` de `iris`.

### 6.1 Dispersión (scatter)

Un diagrama de dispersión relaciona dos variables numéricas; es el gráfico más usado para ver relaciones entre características:

In [25]:

```
plt.scatter(df["sepal_len"], df["petal_len"])
plt.xlabel("Largo de sépalo (cm)")
plt.ylabel("Largo de pétalo (cm)")
plt.title("Iris: sépalo vs pétalo")
plt.show()
```

□ FIGURA

*Nube de puntos de largo de sépalo frente a largo de pétalo.*

### 6.2 Histograma

Un histograma muestra cómo se distribuye una sola variable, agrupando los valores en intervalos (bins):

In [26]:

```
plt.hist(df["sepal_len"], bins=15, color="#4F46E5",
         edgecolor="white")
plt.xlabel("Largo de sépalo (cm)")
plt.ylabel("Frecuencia")
plt.show()
```

□ FIGURA

*Histograma de la distribución del largo de sépalo.*

### 6.3 Barras (con Pandas)

Pandas usa Matplotlib por debajo: el método `.plot()` de una `Series` o `DataFrame` genera el gráfico directamente. Aquí, el largo de pétalo medio por especie:



In [27]:

```
medias = df.groupby("species")["petal_len"].mean()
medias.plot(kind="bar")
plt.ylabel("Largo de pétalo medio (cm)")
plt.show()
```

FIGURA

*Gráfico de barras del largo de pétalo medio por especie.*

## 6.4 Varios ejes (subplots)

plt.subplots crea una figura con varios ejes para comparar gráficos lado a lado. Cada eje se controla por separado:

In [28]:

```
fig, axes = plt.subplots(1, 2, figsize=(9, 3))
axes[0].scatter(df["sepal_len"], df["sepal_wid"])
axes[0].set_title("Sépalo")
axes[1].scatter(df["petal_len"], df["petal_wid"])
axes[1].set_title("Pétalo")
plt.show()
```

FIGURA

*Dos diagramas de dispersión lado a lado: sépalo y pétalo.*

## 07 Seaborn — exploración rápida

Seaborn se construye sobre Matplotlib y trabaja directamente con DataFrames. Con muy poco código produce gráficos estadísticos vistosos, y el parámetro hue colorea automáticamente por categoría. Es la herramienta ideal para el análisis exploratorio (EDA).

### 7.1 Boxplot y dispersión por categoría

Un boxplot compara la distribución de una variable entre grupos; un scatter con hue separa las clases por color:

In [29]:

```
sns.boxplot(data=df, x="species", y="petal_len")
plt.show()
```

FIGURA

*Boxplot del largo de pétalo por especie.*

In [30]:

```
sns.scatterplot(data=df, x="sepal_len", y="petal_len",
                 hue="species")
plt.show()
```

## FIGURA

*Dispersión de sépalo vs pétalo, coloreada por especie.*

## 7.2 Mapa de correlación

El heatmap de correlación resume, de un vistazo, qué variables se mueven juntas. Es uno de los gráficos más usados al comenzar a explorar un dataset:

In [31]:

```
corr = df.drop(columns="species").corr()
sns.heatmap(corr, annot=True, cmap="viridis")
plt.show()
```

## FIGURA

*Mapa de calor de correlaciones entre las medidas de iris.*

## NOTA

Para una exploración completa en una sola línea, `sns.pairplot(df, hue="species")` dibuja todas las combinaciones de variables a la vez. Es más pesado de calcular, pero muy revelador.

## 08 Entregable

Elabora un cuaderno que aplique todo el flujo sobre iris y súbelo al aula virtual.

## EJEMPLO Qué entregar

**Archivo:** GL02\_Apellido\_Nombre.ipynb

1. Celda Markdown inicial con tus datos y un título con formato.
2. NumPy: crea un arreglo, muestra su shape, aplica una operación vectorizada y una agregación con axis.
3. Carga iris en un DataFrame y muestra head(), shape y describe().
4. Aplica un filtrado booleano y un groupby por especie con al menos una columna.
5. Crea una columna derivada nueva a partir de dos columnas existentes.
6. Dos gráficos con Matplotlib (uno de dispersión y otro distinto), con ejes etiquetados.
7. Un gráfico de EDA con Seaborn (boxplot, scatter con hue o heatmap de correlación).
8. El cuaderno debe correr limpio con Restart & Run All antes de entregar.

## 09 Rúbrica de evaluación

| Criterio           | Descripción                                                  | Pts. |
|--------------------|--------------------------------------------------------------|------|
| NumPy              | Arreglo, operación vectorizada y agregación con axis.        | 20   |
| Series y DataFrame | Carga de iris, exploración (head/shape/describe) y filtrado. | 25   |

| Criterio                 | Descripción                                               | Pts. |
|--------------------------|-----------------------------------------------------------|------|
| Transformación           | Columna derivada y groupby por especie coherentes.        | 15   |
| Matplotlib               | Dos gráficos correctos y con ejes etiquetados.            | 20   |
| Seaborn                  | Un gráfico de exploración pertinente.                     | 10   |
| Ejecución y presentación | Corre sin errores (Restart & Run All); cuaderno ordenado. | 10   |

**Total: 100 puntos**

## 10 Referencias

- Raschka, S. & Mirjalili, V. — Python Machine Learning (bibliografía oficial de la asignatura).
- Documentación de NumPy: <https://numpy.org/doc>
- Documentación de Pandas: <https://pandas.pydata.org/docs>
- Documentación de Matplotlib: <https://matplotlib.org/stable>
- Documentación de Seaborn: <https://seaborn.pydata.org>
- Dataset iris en scikit-learn: [https://scikit-learn.org/stable/datasets/toy\\_dataset.html](https://scikit-learn.org/stable/datasets/toy_dataset.html)